

FRED TALKS

7th June 2026

CONTENTS

Highlights from my conversation about agentic engineering on
Lenny's Podcast

SIMON WILLISON'S WEBLOG: ENTRIES · 3349 WORDS

How big are our embeddings now and why?

★♥☆ VICKI BOYKIS ★♥☆ · 1697 WORDS

[Daily article] June 7: Voss (collection)

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 428 WORDS

2.1.168

CHANGELOG · 5 WORDS

Highlights from my conversation about agentic engineering on Lenny's Podcast

SIMON WILLISON'S WEBLOG: ENTRIES · 02 APR 2026 · [SOURCE](#)

Highlights from my conversation about agentic engineering on Lenny's Podcast

I was a guest on Lenny Rachitsky's podcast, in a new episode titled [An AI state of the union: We've passed the inflection point, dark factories are coming, and automation timelines](#). It's available on [YouTube](#), [Spotify](#), and [Apple Podcasts](#). Here are my highlights from our conversation, with relevant links.

[An AI state of the union: We've passed the inflection point & dark factories are coming](#)[Lenny's Podcast](#)



Lenny's Podcast 571K subscribers

[Watch on](#)

- [The November inflection point](#)
- [Software engineers as bellwethers for other information workers](#)
- [Writing code on my phone](#)
- [Responsible vibe coding](#)
- [Dark Factories and StrongDM](#)
- [The bottleneck has moved to testing](#)
- [This stuff is exhausting](#)
- [Interruptions cost a lot less now](#)

- [My ability to estimate software is broken](#)
- [It's tough for people in the middle](#)
- [It's harder to evaluate software](#)
- [The misconception that AI tools are easy](#)
- [Coding agents are useful for security research now](#)
- [OpenClaw](#)
- [Journalists are good at dealing with unreliable sources](#)
- [The pelican benchmark](#)
- [And finally, some good news about parrots](#)
- [YouTube chapters](#)

The November inflection point

4:19—The end result of these two labs throwing everything they had at making their models better at code is that in November we had what I call the [inflection point](#) where GPT 5.1 and Claude Opus 4.5 came along.

They were both incrementally better than the previous models, but in a way that crossed a threshold where previously the code would mostly work, but you had to pay very close attention to it. And suddenly we went from that to... almost all of the time it does what you told it to do, which makes all of the difference in the world.

Now you can spin up a coding agent and say, [build me a Mac application that does this thing](#), and you'll get something back which won't just be a buggy pile of rubbish that doesn't do anything.

Software engineers as bellwethers for other information workers

5:49—I can churn out 10,000 lines of code in a day. And most of it works. Is that good? Like, how do we get from most of it works to all of it works? There are so many new questions that we’re facing, which I think makes us a bellwether for other information workers.

Code is easier than almost every other problem that you pose these agents because code is obviously right or wrong—either it works or it doesn’t work. There might be a few subtle hidden bugs, but generally you can tell if the thing actually works.

If it writes you an essay, if it prepares a lawsuit for you, it’s so much harder to derive if it’s actually done a good job, and to figure out if it got things right or wrong. But it’s happening to us as software engineers. It came for us first.

And we’re figuring out, OK, what do our careers look like? How do we work as teams when part of what we did that used to take most of the time doesn’t take most of the time anymore? What does that look like? And it’s going to be very interesting seeing how this rolls out to other information work in the future.

Lawyers are falling for this really badly. The [AI hallucination cases database](#) is up to 1,228 cases now!

Plus this bit from the cold open at [the start](#):

It used to be you’d ask ChatGPT for some code, and it would spit out some code, and you’d have to run it and test it. The coding agents take that step for you now. And an open question for me is how many other knowledge work fields are actually prone to these agent loops?

Writing code on my phone

8:19—I write so much of my code on my phone. It’s wild. I can get good work done walking the dog along the beach, which is delightful.

I mainly use the Claude iPhone app for this, both with a regular Claude chat session (which [can execute code now](#)) or using it to control [Claude Code for web](#).

Responsible vibe coding

9:55 If you're vibe coding something for yourself, where the only person who gets hurt if it has bugs is you, go wild. That's completely fine. The moment you ship your vibe coding code for other people to use, where your bugs might actually harm somebody else, that's when you need to take a step back.

See also [When is it OK to vibe code?](#)

Dark Factories and StrongDM

12:49 The reason it's called the dark factory is there's this idea in factory automation that if your factory is so automated that you don't need any people there, you can turn the lights off. Like the machines can operate in complete darkness if you don't need people on the factory floor. What does that look like for software? [...]

So there's this policy that nobody writes any code: you cannot type code into a computer. And honestly, six months ago, I thought that was crazy. And today, probably 95% of the code that I produce, I didn't type myself. That world is practical already because the latest models are good enough that you can tell them to rename that variable and refactor and add this line there... and they'll just do it—it's faster than you typing on the keyboard yourself.

The next rule though, is nobody *reads* the code. And this is the thing which StrongDM started doing last year.

I wrote a lot more about [StrongDM's dark factory explorations](#) back in February.

The bottleneck has moved to testing

21:27—It used to be, you'd come up with a spec and you hand it to your engineering team. And three weeks later, if you're lucky, they'd come back with an implementation. And now that maybe takes three hours, depending on how well the coding agents are established for that kind of thing. So now what, right? Now, where else are the bottlenecks?

Anyone who's done any product work knows that your initial ideas are always wrong. What matters is proving them, and testing them.

We can test things so much faster now because we can build workable prototypes so much quicker. So there's an interesting thing I've been doing in my own work where any feature that I want to design, I'll often prototype three different ways it could work because that takes very little time.

I've always loved prototyping things, and prototyping is even more valuable now.

22:40—A UI prototype is free now. ChatGPT and Claude will just build you a very convincing UI for anything that you describe. And that's how you should be working. I think anyone who's doing product design and isn't vibe coding little prototypes is missing out on the most powerful boost that we get in that step.

But then what do you do? Given your three options that you have instead of one option, how do you prove to yourself which one of those is the best? I don't have a confident answer to that. I expect this is where the good old fashioned usability testing comes in.

More on prototyping later on:

46:35—Throughout my entire career, my superpower has been prototyping. I've been very quick at knocking out working prototypes of things. I'm the person who can show up at a meeting and say, look, here's how it could work. And that was kind of my unique selling point. And that's gone. Anyone can do what I could do.

This stuff is exhausting

26:25—I'm finding that using coding agents well is taking every inch of my 25 years of experience as a software engineer, and it is mentally exhausting. I can fire up four agents in parallel and have them work on four different problems. And by like 11 AM, I am wiped out for the day. [...]

There's a personal skill we have to learn in finding our new limits—what's a responsible way for us not to burn out.

I've talked to a lot of people who are losing sleep because they're like, my coding agents could be doing work for me. I'm just going to stay up an extra half hour and set off a bunch of extra things... and then waking up at four in the morning. That's obviously unsustainable. [...]

There's an element of sort of gambling and addiction to how we're using some of these tools.

Interruptions cost a lot less now

45:16—People talk about how important it is not to interrupt your coders. Your coders need to have solid two to four hour blocks of uninterrupted work so they can spin up their mental model and churn out the code. That's changed completely. My programming work, I need two minutes every now and then to prompt my agent about what to do next. And then I can do the other stuff and I can go back. I'm much more interruptible than I used to be.

My ability to estimate software is broken

28:19—I've got 25 years of experience in how long it takes to build something. And that's all completely gone—it doesn't work anymore because I can look at a problem and say that this is going to take two weeks, so it's not worth it. And now it's like... maybe it's going to take 20 minutes because the reason it would have taken two weeks was all of the sort of cruffy coding things that the AI is now covering for us.

I constantly throw tasks at AI that I don't think it'll be able to do because every now and then it does it. And when it doesn't do it, you learn, right? But when it *does* do something, especially something that the previous models couldn't do, that's actually cutting edge AI research.

And a related anecdote:

36:56—A lot of my friends have been talking about how they have this backlog of side projects, right? For the last 10, 15 years, they've got projects they never quite finished. And some of them are like, well, I've done them all now. Last couple of months, I just went through and every evening I'm like, let's take that project and finish it. And they almost feel a sort of sense of loss at the end where they're like, well, okay, my backlog's gone. Now what am I going to build?

It's tough for people in the middle

29:29—So ThoughtWorks, the big IT consultancy, did an offsite about a month ago, and they got a whole bunch of engineering VPs in from different companies to talk about this stuff. And one of the interesting theories they came up with is they think this stuff is really good for experienced engineers, like it amplifies their skills. It's really good for new engineers because it solves so many of those onboarding problems. The problem is

the people in the middle. If you're mid-career, if you haven't made it to sort of super senior engineer yet, but you're not sort of new either, that's the group which is probably in the most trouble right now.

I mentioned [Cloudflare hiring 1,000 interns](#), and Shopify too.

Lenny asked for my advice for people stuck in that middle:

31:21—That's a big responsibility you're putting on me there! I think the way forward is to lean into this stuff and figure out how do I help this make me better?

A lot of people worry about skill atrophy: if the AI is doing it for you, you're not learning anything. I think if you're worried about that, you push back at it. You have to be mindful about how you're applying the technology and think, okay, I've been given this thing that can answer any question and *often* gets it right. How can I use this to amplify my own skills, to learn new things, to take on much more ambitious projects? [...]

33:05—Everything is changing so fast right now. The only universal skill is being able to roll with the changes. That's the thing that we all need.

The term that comes up most in these conversations about how you can be great with AI is *agency*. I think agents have no agency at all. I would argue that the one thing AI can never have is agency because it doesn't have human motivations.

So I'd say that's the thing is to invest in your own agency and invest in how to use this technology to get better at what you do and to do new things.

It's harder to evaluate software

The fact that it's so easy to create software with detailed documentation and robust tests means it's harder to figure out what's a credible project.

37:47 Sometimes I'll have an idea for a piece of software, Python library or whatever, and I can knock it out in like an hour and get to a point where it's got documentation and tests and all of those things, and it looks like the kind of software that previously I'd have spent several weeks on—and I can stick it up on GitHub

And yet... I don't believe in it. And the reason I don't believe in it is that I got to rush through all of those things... I think the quality is probably good, but I haven't spent enough time with it to feel confident in that quality. Most importantly, *I haven't used it yet*.

It turns out when I'm using somebody else's software, the thing I care most about is I want them to have used it for months.

I've got some very cool software that I built that I've *never used*. It was quicker to build it than to actually try and use it!

The misconception that AI tools are easy

41:31—Everyone's like, oh, it must be easy. It's just a chat bot. It's not easy. That's one of the great misconceptions in AI is that using these tools effectively is easy. It takes a lot of practice and it takes a lot of trying things that didn't work and trying things that did work.

Coding agents are useful for security research now

19:04—In the past sort of three to six months, they've started being credible as security researchers, which is sending shockwaves through the security research industry.

See Thomas Ptacek: [Vulnerability Research Is Cooked](#).

At the same time, open source projects are being bombarded with junk security reports:

20:05—There are these people who don't know what they're doing, who are asking ChatGPT to find a security hole and then reporting it to the maintainer. And the report looks good. ChatGPT can produce a very well formatted report of a vulnerability. It's a total waste of time. It's not actually verified as being a real problem.

A good example of the right way to do this is [Anthropic's collaboration with Firefox](#), where Anthropic's security team *verified* every security problem before passing them to Mozilla.

OpenClaw

Of course we had to talk about OpenClaw! Lenny had his running on a Mac Mini.

1:29:23—OpenClaw demonstrates that people want a personal digital assistant so much that they are willing to not just overlook the security side of things, but also getting the thing running is not easy. You’ve got to create API keys and tokens and install stuff. It’s not trivial to get set up and hundreds of thousands of people got it set up. [...]

The first line of code for OpenClaw was written on November the 25th. And then in the Super Bowl, there was an ad for AI.com, which was effectively a vaporware white labeled OpenClaw hosting provider. So we went from first line of code in November to Super Bowl ad in what? Three and a half months.

I continue to love Drew Breunig’s description of OpenClaw as a digital pet:

A friend of mine said that OpenClaw is basically a Tamagotchi. It’s a digital pet and you buy the Mac Mini as an aquarium.

Journalists are good at dealing with unreliable sources

In talking about my explorations of AI for data journalism through Datasette:

1:34:58—You would have thought that AI is a very bad fit for journalism where the whole idea is to find the truth. But the flip side is journalists deal with untrustworthy sources all the time. The art of journalism is you talk to a bunch of people and some of them lie to you and you figure out what’s true. So as long as the journalist treats the AI as yet another unreliable source, they’re actually better equipped to work with AI than most other professions are.

The pelican benchmark

Of course we talked about pelicans riding bicycles:

56:10—There appears to be a very strong correlation between how good their drawing of a pelican riding a bicycle is and how good they are at everything else. And nobody can explain to me why that is. [...]

People kept on asking me, what if labs cheat on the benchmark? And my answer has always been, really, all I want from life is a really good picture of a pelican riding a bicycle. And if I can trick every AI lab in the world into cheating on benchmarks to get it, then that just achieves my goal.

59:56—I think something people often miss is that this space is inherently funny. The fact that we have these incredibly expensive, power hungry, supposedly the most advanced computers of all time. And if you ask them to draw a pelican on a bicycle, it looks like a five-year-old drew it. That's really funny to me.

And finally, some good news about parrots

Lenny asked if I had anything else I wanted to leave listeners with to wrap up the show, so I went with the best piece of news in the world right now.

1:38:10—There is a rare parrot in New Zealand called the Kākāpō. There are only 250 of these parrots left in the world. They are flightless nocturnal parrots—beautiful green dumpy looking things. And the good news is they're having a fantastic breeding season in 2026,

They only breed when the Rimu trees in New Zealand have a mass fruiting season, and the Rimu trees haven't done that since 2022—so there has not been a single baby kākāpō born in four years.

This year, the Rimu trees are in fruit. The kākāpō are breeding. There have been dozens of new chicks born. It's a really, really good time. It's great news for rare New Zealand parrots and you should look them up because they're delightful.

Everyone should [watch the live stream of Rakiura on her nest with two chicks!](#)

YouTube chapters

Here's the full list of chapters Lenny's team defined for the YouTube video:

- 00:00: Introduction to Simon Willison
- 02:40: The November 2025 inflection point
- 08:01: What's possible now with AI coding
- 10:42: Vibe coding vs. agentic engineering
- 13:57: The dark-factory pattern
- 20:41: Where bottlenecks have shifted

- [23:36](#): Where human brains will continue to be valuable
- [25:32](#): Defending of software engineers
- [29:12](#): Why experienced engineers get better results
- [30:48](#): Advice for avoiding the permanent underclass
- [33:52](#): Leaning into AI to amplify your skills
- [35:12](#): Why Simon says he's working harder than ever
- [37:23](#): The market for pre-2022 human-written code
- [40:01](#): Prediction: 50% of engineers writing 95% AI code by the end of 2026
- [44:34](#): The impact of cheap code
- [48:27](#): Simon's AI stack
- [54:08](#): Using AI for research
- [55:12](#): The pelican-riding-a-bicycle benchmark
- [59:01](#): The inherent ridiculousness of AI
- [1:00:52](#): Hoarding things you know how to do
- [1:08:21](#): Red/green TDD pattern for better AI code
- [1:14:43](#): Starting projects with good templates
- [1:16:31](#): The lethal trifecta and prompt injection
- [1:21:53](#): Why 97% effectiveness is a failing grade
- [1:25:19](#): The normalization of deviance
- [1:28:32](#): OpenClaw: the security nightmare everyone is looking past
- [1:34:22](#): What's next for Simon
- [1:36:47](#): Zero-deliverable consulting

- [1:38:05](#): Good news about Kakapo parrots

How big are our embeddings now and why?

★♥☆ VICKI BOYKIS ★♥☆ · 10 SEP 2025 · [SOURCE](#)

A few years ago, I wrote [a paper on embeddings](#). At the time, I wrote that 200-300 dimension embeddings were fairly common in industry, and that adding more dimensions during training would create diminishing returns for the effectiveness of your downstream tasks (classification, recommendation, semantic search, topic modeling, etc.)

I wrote the paper to be resilient to changes in the industry since it focuses on fundamentals and historical context rather than libraries or bleeding edge architectures, but this assumption about embedding size is now out of date and worth revisiting in the context of growing embedding dimensionality and embedding access patterns.

As a quick review, embeddings are compressed numerical representations of a variety of features (text, images, audio) that we can use for machine learning tasks like search, recommendations, RAG, and classification. The size of the embedding is how many features our item has.



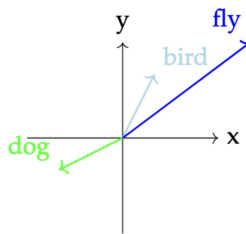
For example, let's say we have two butterflies. We can compare them among many dimensions, including wingspan, wing color, number of antennae. Let's say we have a 3-dimensional feature for a butterfly, it might look like this.

	wing_color	wing_span	antennae
butterfly_1	blue	10 cm	1
butterfly_2	red	20cm	2
butterfly_3	blue	15cm	1

Doing a visual vibe scan, we can eyeball the data and see that `butterfly_1` and `butterfly_3` are more similar to each other than to `butterfly_2` because their features are closer together.

But butterflies are 3-dimensional animals and some of these features are numerical. When we talk about embeddings in industry these days, we generally mean trying to understand the properties of text, so how does this concept work with words? We can't directly compare words like "bird" and "butterfly" or "fly" in a given text but we can compare their numerical representations if we map them into the same shared space. We can see that "bird" and "flying" are more similar to each other than to "dog" through their numerical representations.

Intuitively, we know this is true, but how do we artificially create these relationships?

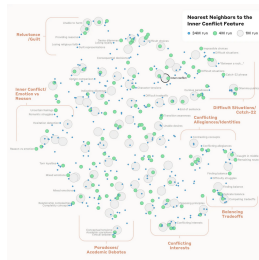


There are different ways of creating text embeddings from a given word, all of which rely on analyzing that word in relation to the words around it in a given corpus of data. We can use traditional count-based approaches like TF-IDF based on term frequency in documents, or statistical approaches like PCA or LSA.

With the advent of deep learning models, we started learning representations generated from models like Word2Vec that maximized the probability that a left-out word would be next to other given words in the training dataset.

When we learn the embedding representation of a given word using probabilistic models, we are comparing how similar these words are to other words. Each feature is not an explicit feature like "wing color", but rather a vibe-based latent representation in the latent space that doesn't have a clear explanation.

For example, one dimension might be "this word is an action word" or maybe "this word is related to other words about food", but we generally don't know exactly what the model thinks each feature represents. In fact, this is a fascinating area of study we are just starting to understand how these latent representations work through ideas like control vectors, a concept that Anthropic explored in the famous Golden Gate Claude paper.



When we train a model, embedding size is initialized as a hyperparameter before model training, and we iterate on the size depending on our downstream evaluations after training. Picking the right hyperparameter is (alchemy) a combination of art and science and depends on optimizing training throughput, final embedding storage size, and the performance of the embedding on your downstream task both qualitatively and wrt to latency of performing search on embeddings of different sizes.

Since previous generations of models were smaller and trained in-house, hyperparameters were usually not published by companies, and as such, there was no standard agreement on embedding size. We generally, as an industry, understood that somewhere around 300 dimensions for a given embedding model might be enough to compress all the nuance of a given textual dataset. 300 was the number of dimensions typically used by earlier models like Word2Vec and GloVE.

After the publication of the attention paper BERT was released in 2018. This model's architecture introduced embeddings of 768 dimensions. Although previous RNN and LSTM models had been trained on GPUs, BERT was one of the first larger embedding models to be trained on GPUs (and TPUs), which meant that GPU optimization now became increasingly important.

The key behind training Transformer models efficiently is the ability to efficiently move data onto GPU and parallelize their matrix multiplication operations between several pipelines, aka attention heads, where each attention head can focus on understanding and defining a different part of the embedding feature space. As such, the embedding needs to be able to be partitioned evenly between the number of attention heads.

BERT has 12 attention heads, so 768 dimensions was selected from a combination of trial and error and efficiently parallelizing computation to “attend” to different parts of the feature space, meaning that, in model training, each head operates on a 64-dimensional subspace of the original 768-dimensional input embedding. This itself comes from the Transformer paper, where each sub-embedding size per head is commonly chosen as 64.

As a result, many BERT-style models, and related model families, used 768 as a baseline for embedding dimensions. Although it had a much larger training dataset, GPT-2 also implemented 768. And, although CLIP uses an embedding size of **768** as derived from the Vision Transformer architecture that CLIP uses for its image encoder, and for consistency, the text encoder also uses this dimension size^[1].

Even though training cycles for BERT were fairly small (4 days for the original BERT model) compared to the months-long pretraining processes that LLMs require these days, it was still hard computationally to infer these embedding sizes, even with GPU optimizations. For BERT, for example,

Finding the most similar pair in a collection of 10,000 sentences req

So in 2019, UKP created and optimized a model, SBERT, focusing on sentence-level representations, which unlocked faster inference, and Minilm became the standard baseline model for document-level chunk embeddings at 384 dimensions and still performs extremely well for a number of tasks.

What else changed was the rise of the open-source HuggingFace platform where model artifacts could be shared. Previously, they were either hosted in undiscoverable repos in GitHub or locked away in scientific repositories missing metadata. The rise of HuggingFace as a platform and the `transformers` library as a centralized API into training and inference for PyTorch-based models unlocked a level of standardization: now, many more people could simply download and replicate the existing models instead of rebuilding arcane research code from scratch. This led to much more standard embedding and architecture sizes used both in academia and industry.

Although 768 held for a fairly long time, with upwards pressure from market competition on model sizes as the result of the release of GPT-2 (the backbone of ChatGPT), we started seeing standard embedding sizes increase.

Part of this was the fact that companies no longer had to infer their own embeddings. Previously, embeddings had been custom-learned in labs or in companies whose core competency was processing large amounts of information that could be culled in retrieval through search or recommendation.

But now, with the advent of ChatGPT and API-based model availability, embeddings became a commodity available with a GET request, and the most popular embeddings became OpenAI's, which used 1536 dimensions, in line with GPT-3, which used much more training data than any previous model. (570GB versus GPT-2 which was 40GB, and 96 attention heads.)

It used to be the case that people only learned or fine-tune their own embeddings, but now all of the major AI providers have their own hosted sets of embeddings. Particularly common ones are OpenAI, with Cohere and Nomic close behind. Google also recently released embeddings for Gemini, with both API and local versions being available.

In addition to standardization via HuggingFace and APIs, MTEB, which benchmarks embeddings publicly, has grown as a resource where you can compare embedding models.

If we look at MTEB, the embedding benchmark today, we can see embedding sizes anywhere from our classic 768 to 4096 and beyond (note all of them are neatly divisible by 2, in line with architectural constraints)

Rank (Desc.)	Model	Zero-shot	Embedding S.
1	gemini-embedding-001	99%	3072
2	Qwen2-Embedding-8B	99%	4096
3	Qwen2-Embedding-4B	99%	2560
4	Qwen2-Embedding-1.5B	99%	1024
5	Llama-Embed-Mistral	99%	4096
6	gte-Qwen2-7B-Instruct	NA	3072
7	multilingual-e5-large-instruct	99%	1024
8	SFR-Embedding-Mistral	96%	4096
9	text-multilingual-embedding-002	99%	768
10	GritLM-7B	99%	4096
11	GritLM-Bv7B	99%	4096

Finally, another consideration has been the change in the landscape with vector databases, which used to be huge, are now becoming more commoditized features in software and platforms like Postgres, S3, and Elasticsearch, leading to less out of the box necessity in vector storage and performance tuning.

With the constant upward pressure on embedding sizes not limited by having to train models in-house, it's not clear where we'll slow down: Qwen-3, along with many others is already at 4096. It does appear that we are beginning to trim down on growth. OpenAI implemented a concept called matryoshka representation learning that trains embeddings with the “most important” concepts first, and additional embeddings adding incremental gains, meaning that an embedding learned in 1024 dimensions might be just as useful in 64, as long as the first 64 dimensions compress most of the information efficiently, and also making sure we re-normalize them. There are also research that indicates that not all embeddings are necessary in retrieval + search tasks, and that we can in fact, in some cases, truncate 50% of them anyway.

It's been fascinating to watch the rise of dimensionality and the constant struggle in tradeoffs between creating ever-larger models and then the need for those embedding sizes to perform at inference and storage time, aka continuously coming up against the age-old machine learning tradeoff of recall versus precision, and the engineering tradeoff of hardware limitations versus software versus business considerations.

As these architectures have matured and internal models have become inference points of public-facing paid APIs, embeddings have gone from a mysterious byproduct of internal machine learning systems in companies with lots of data to a commodity used across many AI-powered applications across numerous application stacks.

All in all, it's been so much fun watching this space evolve, even if it means it looks like I'll be having to update my paper over and over again.

[Daily article] June 7: Voss (collection)

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 07 JUN 2026 · [SOURCE](#)

Voss is the seventeenth collection by British fashion designer Alexander McQueen, made for the Spring/Summer 2001 season of his fashion house. Voss drew on imagery of madness and nature to question beauty standards and critique the fashion industry, with showpiece designs made from unusual materials such as razor clam shells (dress pictured), an antique Japanese screen, and microscope slides. The runway show was staged on 26 September 2000 in London, inside a room-sized mirrored glass cube, the audience seated outside. The interior was styled to look like an insane asylum and models were directed to act unwell. In the finale, a glass cube inside shattered to reveal Michelle Olley, fat, nude, and covered in moths. Critical response was positive, especially towards the showpieces and the performance art aspect. The show is regarded as one of McQueen's best, and has attracted a large amount of academic analysis. Several ensembles appeared in the retrospective Alexander McQueen: Savage Beauty. (Full article...).

Read more: https://en.wikipedia.org/wiki/Voss_%28collection%29

Today's selected anniversaries:

1832:

The Reform Act, which is widely credited with launching modern democracy in the United Kingdom, received royal assent. https://en.wikipedia.org/wiki/Reform_Act_1832

1900:

American temperance activist Carrie Nation entered a saloon in Kiowa, Kansas, and destroyed its stock of alcoholic beverages with rocks. https://en.wikipedia.org/wiki/Carrie_Nation

1929:

The Lateran Treaty was ratified to bring Vatican City into existence, thus ending the "Roman question". https://en.wikipedia.org/wiki/Lateran_Treaty

1989:

Surinam Airways Flight 764 crashed while approaching Johan Adolf Pengel International Airport, in Zanderij, killing 178 people, including members of the football team known as the Colourful 11. https://en.wikipedia.org/wiki/Surinam_Airways_Flight_764

Wiktionary's word of the day:

half-baked: 1. (figurative, frequently derogatory, informal) 2. Of a person or thing: incompletely developed or formed; not fully prepared or trained. 3. Of an idea or scheme; not carefully considered or fully planned; ill- conceived, unsound. 4. Of a person: having no common sense or intelligence; foolish. 5. Of a person: mildly under the influence of alcohol, a drug, etc. 6. Used other than figuratively or idiomatically: partially cooked by heating in an oven, either as an accidental outcome or as a work in process. 7. (informal or UK, dialectal) A foolish or stupid person; a fool, an idiot. 8. About Word of the Day 9. Nominate a word 10. Leave feedback <https://en.wiktionary.org/wiki/half-baked>

Wikiquote quote of the day:

I never meant 2 cause u any sorrowI never meant 2 cause u any painI only wanted 2 one time see u laughingI only wanted 2 see u laughing in the purple rain. --Prince (musician) https://en.wikiquote.org/wiki/Prince_%28musician%29

2.1.168

CHANGELOG · 07 JUN 2026 · [SOURCE](#)

- Bug fixes and reliability improvements